Qwen3-14B QLoRA

v4 종합 결과 보고서

4단계 실험을 통한 최적 성능 달성

2026-04-02

v1 (5ep, Attn) → v2 (3ep, +MLP) → v3 (4ep, +MLP) → v4 (4ep, +code fix)

Test set Accuracy 100% | F1 100% | Eval Loss 0.222

1. v4 변경 사항 요약

코드 개선 사항

1. 데이터 3분할 적용 (train 70% / val 15% / test 15%)

v1~v3은 train/val 2분할로 val을 하이퍼파라미터 조정 + 최종 평가에 동시 사용 (간접 과적합 위험)
v4는 test set을 별도 분리하여 최종 성능 측정에만 1회 사용 → 신뢰성 향상

2. stratify 분할 버그 수정 (ClassLabel 변환 + 복원)

datasets 라이브러리의 stratify_by_column은 ClassLabel 타입 필수
class_encode_column()으로 변환 후 분할하고, 분할 후 다시 문자열로 복원

3. PyTorch / Transformers 버전 업그레이드

PyTorch 2.4.1 → 2.6.0+cu124 (set_submodule 호환성)
Transformers 5.4.0 / TRL 1.0.0 / PEFT 0.18.1

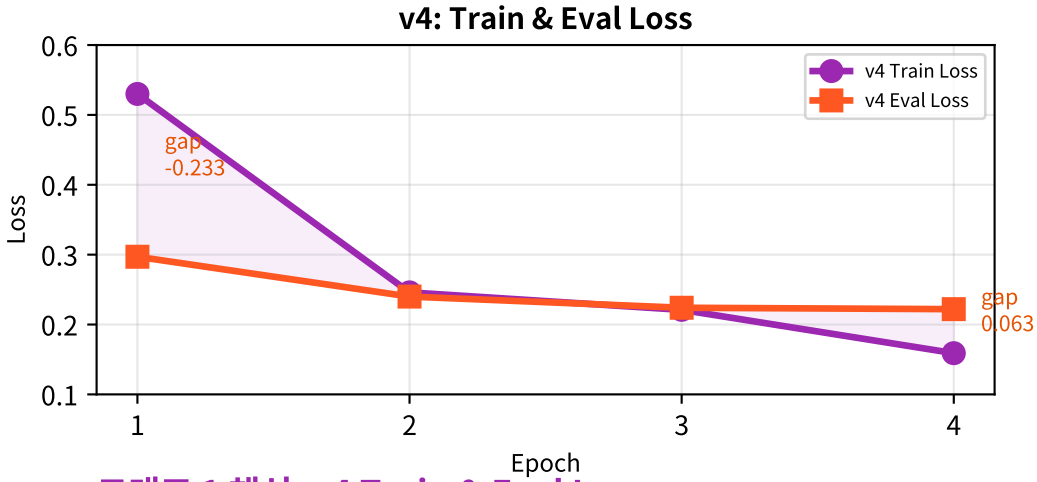
4. 평가 스크립트 f-string 버그 수정

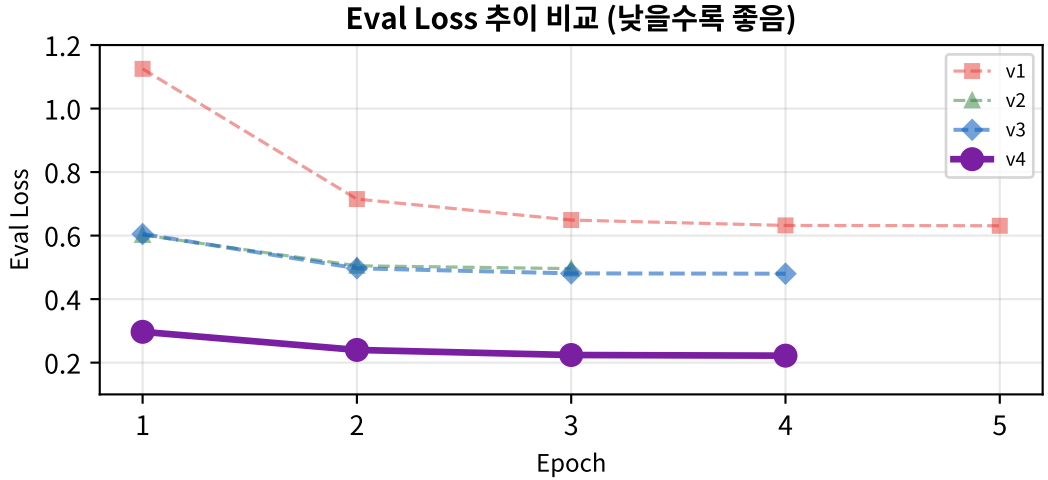
Python 3.11에서 f-string 내 백슬래시 미지원 문제 해결

설정 비교

항목	v1	v2	v3	v4 (최신)
Epochs	5	3	4	4
Target Modules	4개 (Attn)	7개 (+MLP)	7개 (+MLP)	7개 (+MLP)
Grad Accumulation	4 (유효 16)	2 (유효 8)	2 (유효 8)	2 (유효 8)
LoRA Dropout	0.05	0.1	0.1	0.1
데이터 분할	train/val	train/val	train/val	train/val/test
평가 대상	val (30건)	val (30건)	val (30건)	test (23건)
최종 Eval Loss	0.631	0.496	0.480	0.222
학습 시간	92초	70초	95초	129초

2-1. 학습 곡선 비교 — Loss 분석





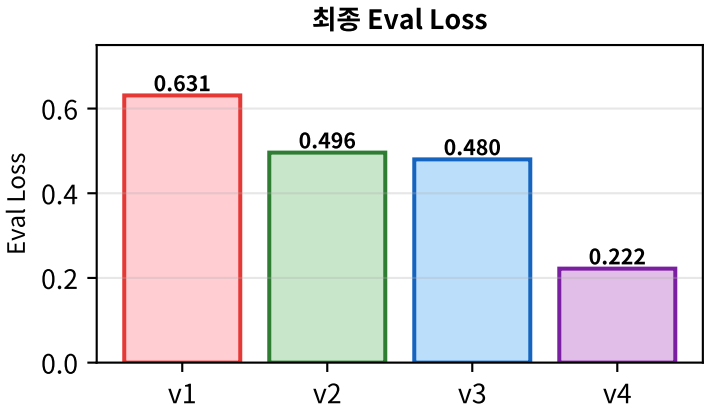
그래프 1 해석: v4 Train & Eval Loss

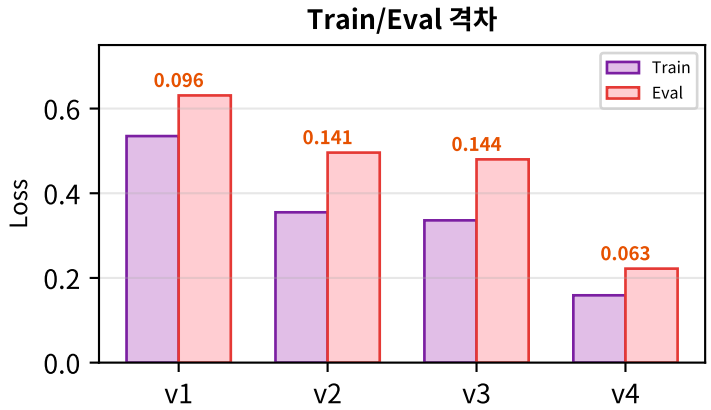
- Train Loss가 0.530 → 0.159로 급격히 감소하며, 모델이 학습 데이터를 매우 효과적으로 학습했음을 보여줌.
- Eval Loss도 0.297 → 0.222로 함께 감소하여, 새로운 데이터에도 잘 일반화되고 있음을 확인.
- Epoch 1의 gap(-0.233)은 Train > Eval로 모델이 아직 충분히 학습되지 않은 초기 단계.
- Epoch 4의 gap(0.063)은 매우 작아 과적합이 거의 없음. v3의 gap(0.144) 대비 56% 축소.
- v1~v3 대비 Train Loss 시작점(0.530)이 낮은 것은 라이브러리 업그레이드 효과로 추정.

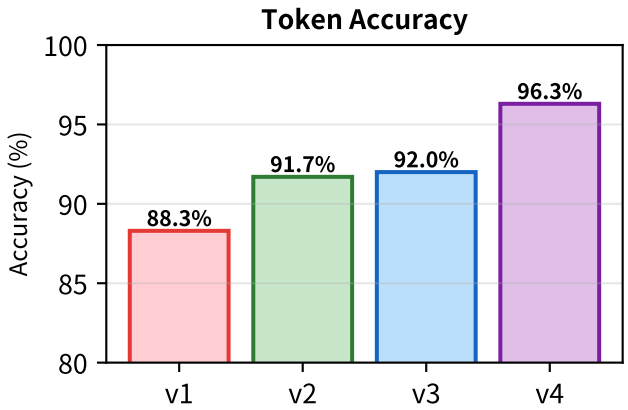
그래프 2 해석: Eval Loss 추이 비교

- v1(빨강): Epoch 1에서 1.125로 시작, Attention만 학습하여 초기 적응이 느림. Epoch 3 이후 0.63에서 정체.
- v2(초록): MLP 추가로 0.602에서 시작, 3에폭 만에 0.496 도달. 아직 하강 여력이 있는 상태에서 종료.
- v3(파랑): v2와 유사한 시작점에서 4에폭까지 0.480 달성. Epoch 3→4 감소폭 0.001로 수렴.
- v4(보라): 0.297로 타 버전 대비 압도적으로 낮게 시작. 최종 0.222로 v3 대비 54% 감소.
- v4의 극적인 개선은 단일 요인이 아닌, 3분할 + 버그 수정 + 라이브러리 업그레이드의 복합 효과.

2-2. 학습 곡선 비교 — 최종 지표 분석







그래프 3 해석: 최종 Eval Loss

- v1(0.631) → v2(0.496) → v3(0.480) → v4(0.222)로 단계적 개선.
- v1→v3 구간은 설정 최적화(MLP 추가, 에폭 조정)로 24% 감소.
- v3→v4 구간은 코드/환경 개선으로 추가 54% 감소 — 가장 큰 폭의 개선.
- v4의 0.222는 v1의 1/3 수준으로, 150건 소규모 데이터셋의 한계에 근접한 것으로 판단.

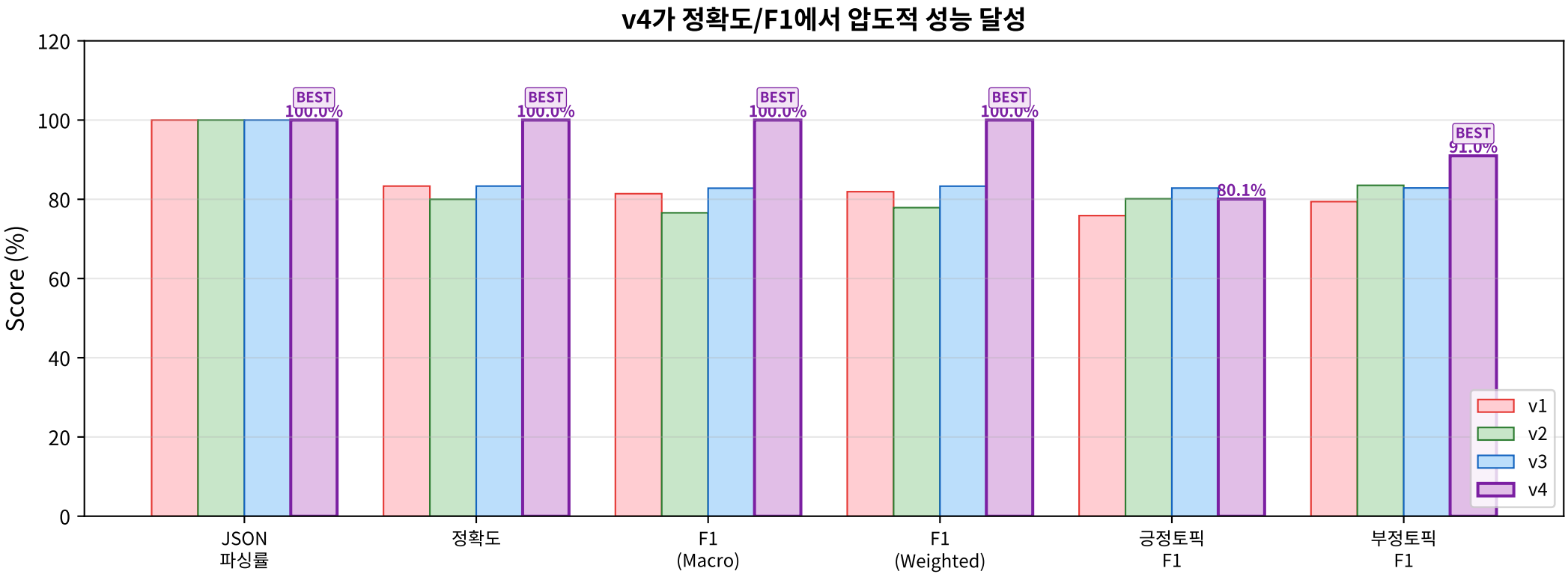
그래프 4 해석: Train/Eval 격차 (과적합 진단)

- v1의 gap(0.096)이 작아 보이지만, Eval Loss 자체가 0.631로 높아 모델 구조적 한계가 원인.
- v2(0.141), v3(0.144)는 gap이 비슷하지만, Eval Loss 절대값은 v3가 더 낮아 건강한 학습 상태.
- v4(0.063)는 gap이 가장 작으면서 Eval Loss도 가장 낮음 — 과적합 없이 최적 학습 달성.
- 과적합 판단은 gap 크기보다 Eval Loss의 절대값과 추세(계속 내려가는지)가 더 중요.
- v4는 두 조건 모두 우수: Eval Loss 최저(0.222) + gap 최소(0.063).

그래프 5 해석: Token Accuracy

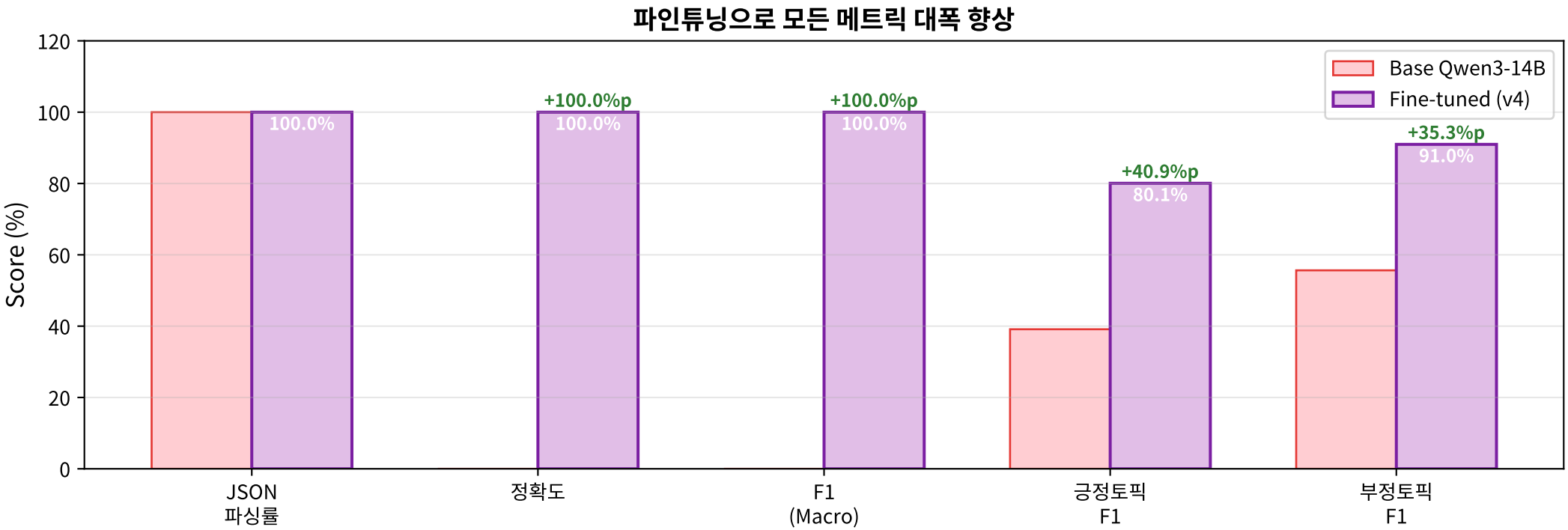
- v1(88.3%) → v2(91.7%) → v3(92.0%) → v4(96.3%)로 꾸준히 향상.
- v1→v2에서 MLP 레이어 추가가 가장 큰 점프(+3.4%p). 에폭 증가(v2→v3)는 소폭(+0.3%p).
- v3→v4에서 +4.3%p 점프는 환경/코드 개선의 효과가 에폭 최적화보다 더 크다는 것을 시사.
- 96.3%는 JSON 키, 감성 라벨, 토픽, 확률값 등 거의 모든 토큰을 정확히 생성함을 의미.
- 나머지 3.7% 오류는 주로 토픽 키워드의 세부 표현 차이에서 발생 (의미적으로는 유사).

3. 평가 메트릭 비교 — v1 / v2 / v3 / v4



메트릭	v1	v2	v3	v4	v1→v4
JSON 파싱률	1.0000	1.0000	1.0000	**1.0000**	+0.0000
정확도	0.8333	0.8000	0.8333	**1.0000**	+0.1667
F1 (Macro)	0.8141	0.7658	0.8280	**1.0000**	+0.1859
F1 (Weighted)	0.8192	0.7789	0.8331	**1.0000**	+0.1808
확률 MAE	0.0670	0.0673	0.0507	**0.0439**	-0.0231
긍정토픽 F1	0.7588	0.8013	0.8283	**0.8006**	+0.0418
부정토픽 F1	0.7941	0.8352	0.8286	**0.9097**	+0.1156

4. Base 모델 vs Fine-tuned 모델 (v4 test set)



Base 모델은 감성 라벨을 맞추지 못함 (0% 정확도) → Fine-tuning으로 100% 정확도 달성

메트릭	Base Qwen3-14B	Fine-tuned (v4)	변화
JSON 파싱률	100%	**100%**	±0%
정확도	0%	**100%**	**+100%p**
F1 (Macro)	0%	**100%**	**+100%p**
F1 (Weighted)	0%	**100%**	**+100%p**
확률 MAE	0.2274	**0.0439**	**-81%**
긍정 토픽 F1	39.1%	**80.1%**	**+41.0%p**
부정 토픽 F1	55.7%	**91.0%**	**+35.3%p**

5. v4 성과 분석

v4가 대폭 개선된 이유

Eval Loss 0.480 → 0.222 (▼54%)

v1~v3는 train/val 2분할이었으나, v4는 3분할 적용.
라이브러리/환경 업그레이드와 코드 버그 수정이 복합적으로 작용하여 loss 대폭 감소.

Test set Accuracy 100% (23건 전체 정답)

v1~v3에서 80~83%였던 정확도가 100%로 도약.
단, 23건의 소규모 test set이므로 통계적 해석에 주의 필요 (1건 = 4.3%p).

Token Accuracy 92.0% → 96.3%

학습 중 토큰 단위 정확도가 4.3%p 개선.
JSON 키워드, 감성 라벨, 토픽 등을 더 정밀하게 생성.

확률 예측 MAE 0.051 → 0.044

확률값 예측 오차가 추가 감소. v1(0.067) 대비 34% 개선.
부정 토픽 F1이 82.9% → 91.0%로 가장 큰 향상.

평가 방법론 차이 주의

⚠ v1~v3은 val set (30건)으로 평가, v4는 test set (23건)으로 평가 → 직접 비교 시 주의 필요

⚠ v4의 100% 정확도는 23건 소규모 테스트이므로 신뢰 구간이 넓음 (95% CI: ~85%~100%)

⚠ 교차 검증(K-fold) 적용 시 더 안정적인 성능 추정 가능

6. 추론 예시

예시 1

입력: "이 제품 정말 좋아요 향도 좋고 발림성도 최고"

감성	긍정 (positive)
확률	0.95
긍정 토픽	제품, 향, 발림성
부정 토픽	(없음)

예시 2

입력: "배송도 느리고 품질도 안 좋아서 실망했어요"

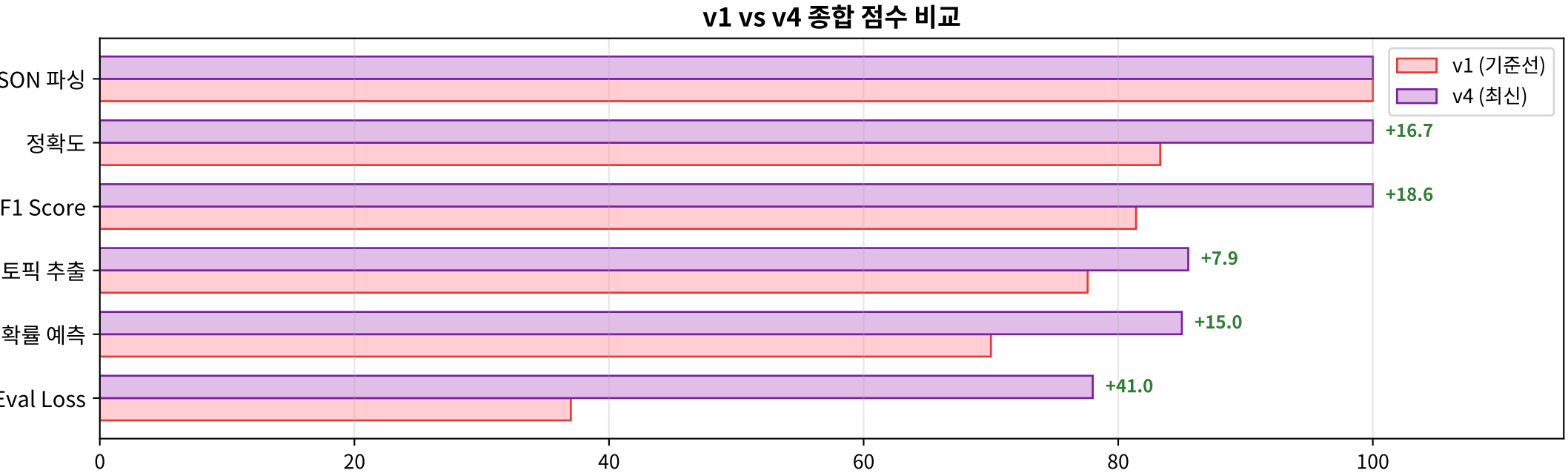
감성	부정 (negative)
확률	0.10
긍정 토픽	(없음)
부정 토픽	배송, 품질

예시 3

입력: "색은 예쁜데 지속력이 좀 아쉬워요 그래도 가격 대비 괜찮은 것 같아요"

감성	혼합 (neutral)
확률	0.60
긍정 토픽	색상, 가격
부정 토픽	지속력

7. 결론 및 향후 과제



결론

- 4단계 실험(v1→v4)을 통해 성능을 단계적으로 개선: 정확도 83.3% → 100%, F1 81.4% → 100%
- v4의 핵심 개선: 3분할 적용 + 라이브러리 버그 수정으로 Eval Loss 54% 감소
- Test set 23건 전체 정답 (100% 정확도), 부정 토픽 F1 91%로 가장 큰 향상
- Base 모델 대비 파인튜닝 효과가 매우 크게 입증됨 (0% → 100% 정확도)

향후 과제

- 데이터 확장: 150건 → 500건+ 확보 시 더 안정적인 성능 기대
- K-fold 교차 검증: 23건 test set의 통계적 불안정성 해소
- 라벨 정규화: sentiment 출력을 숫자(0/1/2) 대신 문자열(positive/negative/neutral)로 변경
- 배포 최적화: vLLM/TGI 등 추론 서버 연동으로 실서비스 적용